

Course Introduction

Introduzione

Definizione: Event-Driven Architecture (architettura basata sugli eventi)

È un **modello di architettura software in cui il flusso del sistema è definito dagli eventi**, non dalle chiamate dirette tra componenti.

Definizione di "evento": un'occorrenza significativa, un cambiamento di stato, o una notifica che qualcosa è accaduto dentro un servizio o nel sistema più ampio. Esempi concreti dal dominio e-commerce del corso:

- OrderPlacedEvent (ordine inserito)
- ProductPriceChangedEvent (prezzo prodotto cambiato)
- UserRegisteredEvent (utente registrato)
- InventoryUpdatedEvent (inventario aggiornato)

Il meccanismo chiave: in questa architettura un servizio **non chiama direttamente** un altro servizio in modo sincrono. Invece:

1. Esegue la sua azione (es. salva l'ordine nel DB)
2. **Pubblica un evento** che dichiara "questo è successo"
3. Chi è interessato si è **sottoscritto** a quel tipo di evento e lo riceve
4. L'evento passa per un **event bus / message broker** che lo distribuisce

Il risultato è un **accoppiamento molto debole (loose coupling)**: i servizi non devono conoscersi tra loro, devono conoscere solo il *formato degli eventi* che producono o consumano. La comunicazione è per natura **asincrona**.

Perché serve: il problema che risolve è quello dei microservizi fortemente inter dipendenti. Se il servizio Ordini deve chiamare a mano Inventario + Notifiche + Spedizioni e aspettare le risposte, ha creato tre dipendenze rigide su di sé.

Definizione: Pattern Publish/Subscribe (Pub/Sub)

È **il meccanismo concreto che abilita l'event-driven architecture**. Ha tre componenti:

1. Publisher (editore)

Il servizio che genera e invia l'evento. Punto cruciale: il publisher invia l'evento a un **canale centrale** (chiamato *topic* o *event bus*) **senza sapere chi sono gli iscritti, né se esistono iscritti**. Si limita ad annunciare che qualcosa è accaduto.

2. Subscriber (iscritto)

Il servizio interessato a uno specifico tipo di evento. Registra il proprio interesse **iscrivendosi a un topic**. Quando un evento viene pubblicato su quel topic, ne riceve una **copia** che poi consuma ed elabora.

Esempio della trascrizione: sia il servizio Notifiche sia il servizio Inventario possono essere sottoscritti allo stesso evento "ordine inserito".

3. Message Broker / Event Bus

Il componente infrastrutturale intermedio che gestisce i topic e si occupa della **consegna** degli eventi dai publisher a tutti i subscriber interessati. È lui che **disaccoppia fisicamente** publisher e subscriber.

Modello di diffusione (fan-out): qualsiasi messaggio pubblicato su un topic viene ricevuto da **tutti** i subscriber attivi su quel topic. Un singolo evento viene quindi distribuito simultaneamente a più consumatori.

I 4 vantaggi del Pub/Sub

Vantaggio	Cosa significa concretamente
Disaccoppiamento	Puoi modificare, aggiornare o aggiungere un subscriber senza toccare il publisher e senza che il publisher ne sia a conoscenza. L'unico contratto è il formato dell'evento.
Scalabilità	Publisher, subscriber e broker scalano in modo indipendente . Se hai troppi eventi di un tipo, aumenti il numero di subscriber per elaborarli in parallelo. <i>Esempio citato: KEDA</i> , l'autoscaler event-driven per Kubernetes, scala automaticamente i microservizi in base al carico del message broker.
Flessibilità / resilienza	Se un subscriber è temporaneamente down, il broker accoda gli eventi. Quando torna online li smaltisce. Questo evita che un guasto a valle propaghi l'interruzione.
Reattività (velocità di risposta)	Chi pubblica l'evento non aspetta che i subscriber lo elaborino: pubblica e passa oltre. L'operazione percepita dall'utente è molto più veloce.

Esempio completo dal corso: quando viene effettuato un ordine, il microservizio Ordini pubblica un evento `OrderConfirmed`. I microservizi **Inventario**, **Notifiche** e **Spedizioni** sono tutti sottoscritti e svolgono i rispettivi compiti in modo **asincrono e parallelo**. Il servizio Ordini **non** deve fare tre chiamate separate e attendere il completamento di ciascuna.

Le 5 sfide / considerazioni del Pub/Sub

1. **Il message broker è un componente critico** — può diventare un *single point of failure* se non è distribuito in configurazione altamente disponibile. Anche le sue prestazioni e la sua scalabilità diventano un vincolo del sistema.
2. **Debug e tracing dei flussi distribuiti** — poiché un evento coinvolge più servizi che agiscono indipendentemente, tracciare il flusso end-to-end è molto più complesso rispetto a una catena di chiamate sincrone. Serve **tracing e logging distribuiti** (argomento della Lezione 15).
3. **Eventual consistency (coerenza finale)** — poiché i servizi gestiscono gli eventi in modo asincrono, i dati tra microservizi diventano coerenti **col tempo**, non immediatamente dopo la pubblicazione. Sistema e UX vanno progettati sapendolo (es. "il tuo ordine è in elaborazione" invece di mostrare subito lo stato definitivo).
4. **Garanzia dell'ordine dei messaggi** — alcuni broker garantiscono l'ordine solo **all'interno di una partizione** o di una specifica coda; l'ordine globale tra tutti i consumer di solito **non** è garantito. Se l'ordine stretto è critico, serve una progettazione attenta.
5. **Idempotenza dei consumer** — per problemi di rete o meccanismi di *retry* del broker, **un consumer può ricevere lo stesso evento più di una volta**. I consumer devono quindi essere **idempotenti**: elaborare lo stesso evento N volte deve avere lo stesso effetto di elaborarlo una volta sola.

Esempio pratico: se ricevi due volte `OrderPlaced(orderId=123)`, non devi creare due ordini. Tieni traccia degli `eventId` già processati e scarta i duplicati.

Architecture Design Journey: Evolution of Architectures

Questa sezione definisce **il percorso che il corso seguirà**.

Il contesto storico

Negli anni il panorama dello sviluppo software è cambiato profondamente:

- Si partiva (e molti ancora lavorano) con **architetture monolitiche tradizionali**: tutti i componenti strettamente interconnessi e distribuiti come **un'unica unità**.
- Con l'aumentare di complessità e necessità di scala, il codice monolitico ha iniziato a mostrare problemi su **velocità di sviluppo, distribuzione e scalabilità**.
- Il settore è quindi passato ad **applicazioni distribuite**, spinto dall'avvento del **cloud-native**.
- L'evoluzione è proseguita verso architetture **function-focused** e **event-driven**.

Le 5 tappe del corso

Il corso costruirà sistematicamente l'architettura di un'applicazione e-commerce attraverso queste fasi:

1. **Architettura monolitica** — il punto di partenza. Vantaggi strutturali per applicazioni semplici, ma con un **limite intrinseco quando si deve scalare**.
2. **Monolite modulare** — ristrutturazione che porta **migliore organizzazione interna**, mantenendo però **un unico processo di deploy**.
3. **Architettura a microservizi** — il salto grande. Qui si dedica molto tempo a **scomporre l'interconnessione** e progettare **servizi autonomi**.
4. **Microservizi event-driven** — miglioramento dei microservizi per renderli realmente guidati dagli eventi, ottenendo maggiore separazione, flessibilità e reattività.
5. **Serverless microservices** — valutazione di come il **FaaS (Function as a Service)** si integri nel design complessivo, per casi d'uso specifici.

Durante l'evoluzione, il focus si sposta **dai vincoli dell'infrastruttura on-premise alle possibilità del cloud-native e del serverless**, e l'obiettivo dichiarato è **riprogettare il sistema per gestire milioni di richieste**.

La stessa evoluzione vista come "riduzione di dimensione dei servizi"

C'è una seconda prospettiva interessante: il percorso è essenzialmente un cammino **dal grande al piccolo**.

- **Monoliti** — tutte le funzioni di business raggruppate e distribuite insieme, di solito condividendo **un unico grande database**. In questa categoria rientrano sia i monoliti classici sia i monoliti modulari.
- **Microservizi** — molto più piccoli. Ogni microservizio è progettato attorno a **una specifica funzionalità di business**. Sviluppati, distribuiti e gestiti in modo **indipendente**, comunicano tra loro in modo disaccoppiato.

- **Nanoservizi / funzioni serverless** — ancora più granulari. Progettati per svolgere **una singola attività o funzione specifica**, esposta tramite un endpoint API dedicato.

La domanda cruciale dell'architetto

"Quale approccio architetturale dovremmo adottare? Monolite, microservizi o serverless?"

La risposta della lezione è netta: **non esiste una risposta predefinita né una soluzione migliore in assoluto**. La scelta ottimale dipende sempre dai requisiti specifici del progetto. I fattori che pesano:

- dimensione e competenze del team
- complessità del problema
- requisiti di scalabilità
- pressione sul *time-to-market*
- infrastruttura esistente

Ed è **molto comune e spesso molto efficace un approccio ibrido o di transizione**. Due esempi citati:

- una piattaforma che resta parzialmente monolitica per garantire stabilità, mentre le **nuove funzionalità** vengono sviluppate come microservizi;
- un'architettura prevalentemente a microservizi che **sfrutta funzioni serverless** per attività specifiche e basate su eventi.

Le "considerazioni verticali" (i pilastri trasversali)

Questo è un concetto chiave che ritornerà per tutto il corso. Mentre le architetture sono l'asse "orizzontale" (monolite → microservizi → serverless), le **considerazioni verticali** sono i pilastri fondamentali che attraversano qualsiasi architettura a microservizi robusta. Sono 6:

#	Pilastro	Domanda che risolve	Lezioni
1	Decomposizione	Come spezzare il monolite? Come definire i confini dei servizi ?	Lez. 6
2	Comunicazione	Sincrona (REST, gRPC) o asincrona (messaggi)? Qual è il ruolo degli API Gateway ?	Lez. 7, 8, 9
3	Gestione dei dati	Come gestisce ciascun servizio il proprio database? Come si fanno le query che coinvolgono più servizi ?	Lez. 10, 11
4	Transazioni distribuite	Come gestire una transazione che attraversa più servizi? (pattern Saga)	Lez. 12
5	Deployment	Come impacchettare, distribuire e gestire i servizi con container e orchestratori ?	Lez. 14
6	Resilienza e osservabilità	Come costruire sistemi resilienti ai guasti ? Come monitorare log e tracciare richieste tra servizi distribuiti?	Lez. 15

A ciascuno di questi pilastri verranno applicati design pattern e best practice, con ristrutturazioni continue dell'architettura e **esempi software concreti**.

Way of Learning: The Course Flow – Problem → Problem → Learn → Design → Code → Eval

Questa sezione spiega la **metodologia didattica**, ed è il cuore di come è organizzato tutto il corso.

Il principio di fondo

L'architettura si fonda sulla risoluzione dei problemi.

Quindi il corso non presenta i pattern come un catalogo, ma li fa **emergere da un problema concreto**. La struttura ruota attorno a un **ciclo di apprendimento iterativo** di 5 passi, ripetuto per quasi ogni argomento principale.

I 5 passi del ciclo

1. PROBLEM (Problema)

Si parte sempre definendo e comprendendo chiaramente un problema o una sfida specifica. Può essere:

- un **requisito di business** (es. "dobbiamo gestire più utenti")
- una **limitazione tecnica** dell'architettura attuale (es. deployment lenti, colli di bottiglia di scalabilità)

2. LEARN (Apprendimento)

Una volta chiaro il problema, si esplorano gli **approcci architetturali, design pattern, principi e best practice** rilevanti *per quel problema specifico*. È la fase teorica: si capisce quali strumenti sono disponibili nel proprio "toolkit architetturale".

3. DESIGN (Progettazione)

Con quelle conoscenze si **progetta una soluzione**: si disegnano diagrammi, si valutano i **trade-off (compromessi)** e si crea un piano architetturale che affronti il problema.

4. CODE (Codice)

È ciò che rende particolare questo corso. Si vedono **esempi di codice .NET concreti** relativi al negozio online: si esplora il codice sorgente, si esegue l'applicazione, si vedono i pattern in azione. Serve a **consolidare e visualizzare le implicazioni pratiche** delle scelte di progettazione.

5. EVAL (Valutazione)

Dopo aver progettato e visto il codice si valuta il nuovo stato architetturale:

- Quali sono i suoi punti di forza e di debolezza?
- Come ha risolto il problema iniziale?
- **Quali nuove sfide ha introdotto?**

E qui sta il meccanismo iterativo: **la valutazione genera il problema successivo**, che fa ripartire il ciclo.

Esempio del ciclo in azione (il primo giro)

Passo

Contenuto concreto

Problem "Abbiamo bisogno di un modo semplice per vendere prodotti online"

Passo	Contenuto concreto
Learn	Si studia il dominio e-commerce, si identificano requisiti funzionali e non funzionali, si esplora il modello architetturale di base più adatto: l'architettura monolitica
Design	Si progetta l'architettura iniziale come monolite
Code	Si analizza il codice .NET del monolite, se ne studia la struttura, si esegue una versione semplificata
Eval	Vantaggi e svantaggi? Si identificano i punti critici: <i>cosa succede quando il traffico aumenta significativamente? come gestiamo le parti dell'app che devono scalare in modo indipendente?</i>
→ Quelle domande finali diventano il problema della lezione successiva.	

Perché questa metodologia conta

La trascrizione dà tre motivi espliciti:

1. **Comprensione profonda del contesto** — non impari solo *cos'è* un pattern, ma **perché** viene usato per risolvere un problema specifico.
2. **Applicazione pratica** — non è un corso puramente teorico; i principi si vedono applicati nel codice.
3. **Mentalità dei trade-off** — l'architettura si basa su **compromessi e perfezionamenti iterativi**. *Raramente esiste una soluzione perfetta e definitiva.*

La mappa completa Problema → Soluzione del corso

Questa lista è di fatto **l'indice ragionato dell'intero corso**, ed è utilissima da tenere sott'occhio:

Problema	Soluzione che verrà appresa	Lezione
Accoppiamento stretto tra UI e backend	Architettura a tre livelli (3-tier)	2
Difficoltà di gestire ed evolvere un monolite	Architettura modulare (modular monolith)	3
Colli di bottiglia su larga scala	Passaggio ai microservizi	4, 5
Come decomporre efficacemente in microservizi	Pattern di decomposizione	6
Come connettere efficacemente i microservizi	Protocolli di comunicazione, versioning delle API , service communication	7
Comunicazione diretta client → servizio	API ben definite e pattern di comunicazione	7, 8
Client diversi con esigenze di UI diverse	Pattern BFF (Backend For Frontend) e API Gateway	8
Operazioni a lunga durata non gestibili in sincrono	Comunicazione asincrona basata su messaggi	9
Il database diventa il collo di bottiglia in scala	Tipi di database diversi e pattern di partizionamento/sharding	10
Query cross-service e separazione comandi/query	Microservices data & query patterns (CQRS)	11
Gestione delle transazioni distribuite	Pattern Saga e transazioni esterne	12
Costo dell'archiviazione ridondante / letture costose	Cache distribuita	13
Distribuzione e gestione dei	Container e orchestratori	14

Prerequisites, Source Code and Course Slides

La natura del corso (punto importante)

Viene sottolineato esplicitamente: **è un corso di software engineering design, non un corso di sviluppo passo-passo.**

- L'obiettivo primario è **apprendere pattern, principi e pensiero architetturale**, e imparare a prendere decisioni di progettazione consapevoli.
- Il codice serve a **dare vita alla progettazione architetturale**, non è il fine.
- **Non** si costruiscono applicazioni da zero riga per riga. Invece si **esamina e analizza codice già completo** che riflette le architetture progettate: si vedono i sistemi in azione, si fa **debug di parti specifiche** per capire il flusso e l'interazione tra componenti, si testa l'applicazione per osservarne il comportamento e come soddisfa gli obiettivi architetturali.

Prerequisiti

Per seguire le **decisioni di progettazione**, i pattern e i principi: **nessun prerequisito specifico**. Contano i concetti.

Se invece vuoi **eseguire i repository localmente** (fortemente consigliato per una comprensione più profonda), ti servono tre strumenti:

Strumento	Versione	A cosa serve
.NET SDK	.NET 9	Include già .NET Aspire di default, usato per l'orchestrazione di applicazioni distribuite
Visual Studio 2022 (o successivo)	—	IDE / ambiente di sviluppo
Docker Desktop	—	Eseguire facilmente i servizi di supporto con cui interagiscono i microservizi: database, cache e message broker

Codice sorgente

Tutto il codice è su GitHub:

<https://github.com/mehmetozkaya/Design-Microservices-Architecture-with-Patterns-Principles.git>

Il repository è **organizzato in cartelle**, una per ogni stato evolutivo dell'applicazione, così puoi trovare il codice pertinente a ciascuna sezione o pattern architetturale. Nel tuo zip ce l'hai già dentro la cartella della Lezione 1, e la struttura è proprio quella del percorso descritto sopra:

1-eshop-monolith-one-app	→ monolite iniziale
2-eshop-monolith-add-db	→ monolite + database
3-eshop-monolith-add-ui	→ monolite 3-tier (UI + ApiService)
4-eshop-modular-monoliths-first	→ monolite modulare (Catalog/Basket/Ordering)

5-eshop-modular-monoliths-add-cache→ modular monolith + cache
6-eshop-microservices-first → primi microservizi
7-eshop-microservices-sync-call → comunicazione sincrona tra servizi
8-eshop-microservices-yarp-api-gw → API Gateway con YARP
9-eshop-microservices-async-rabbitmq → comunicazione asincrona con RabbitMQ
10-eshop-microservices-hybrid-cache→ cache distribuita
11-eshop-microservices-outbox → pattern Outbox

Nota come la numerazione delle cartelle **ricalchi esattamente l'evoluzione architetturale** del corso: è la prova visiva del "percorso" descritto nella Sezione 2.

Le slide sono scaricabili dalla sezione **Risorse** di Udemy associata alla lezione (nel tuo zip hai già `designv3+slides.pdf`).

First Problem : Sell Products Online – Understand E-Commerce Domain

Qui parte ufficialmente il **primo giro del ciclo**, con il passo **PROBLEM**.

Il problema

"Vendere prodotti online."

È il motore principale del business. L'obiettivo: creare un'applicazione web di e-commerce che permetta ai clienti di **navigare, selezionare e acquistare prodotti**.

Osservazione importante della lezione: *sembra ovvio e semplice, ma anche questa formulazione iniziale comporta una serie di prerequisiti che costituiranno la base della progettazione architetturale iniziale*. Per capire il problema bisogna capire il **dominio e-commerce**, i suoi requisiti e i casi d'uso.

Requisiti funzionali (cosa deve fare il sistema)

Definizione: i requisiti funzionali descrivono **le funzionalità e i comportamenti** che il sistema deve offrire — cosa fa.

Come minimo l'applicazione deve consentire agli utenti di:

1. **Navigare e visualizzare elenchi di prodotti** (browse product list)
2. **Vedere i dettagli di un singolo prodotto** (product details)
3. **Aggiungere prodotti al carrello** (add to cart)
4. **Effettuare un ordine** per tali prodotti (checkout / place order)

Nota: questi quattro casi d'uso corrispondono **esattamente** ai bounded context che diventeranno i microservizi del corso: **Catalog** (1 e 2), **Basket** (3), **Ordering** (4). Lo vedrai nelle cartelle del codice sorgente.

Requisiti non funzionali (come deve comportarsi il sistema)

Definizione: i requisiti non funzionali — chiamati anche "**capabilities**" nella lezione — descrivono **la qualità del servizio**: non *cosa* fa il sistema, ma *quanto bene* lo fa. Sono quelli che guidano davvero le decisioni architetturali.

Requisito	Definizione	Cosa implica concretamente
Disponibilità (Availability)	Il negozio deve essere "aperto" 24/7 . Almeno: deve essere altamente disponibile .	Se vendi online e il sito è giù, non incassi. Porterà a ridondanza, health check, failover.
Performance degli ordini (Throughput)	Deve gestire un numero ragionevole di ordini al secondo .	<i>"Non può gestire solo cinque utenti."</i> Il valore di "ragionevole" verrà definito numericamente più avanti — è proprio il calcolo che si farà nella Lezione 4 sulla scalabilità .
Tempo di risposta (Latency)	Le pagine devono caricarsi rapidamente e le azioni devono apparire reattive.	Porterà a caching (Lez. 13), CDN, ottimizzazione query.
Affidabilità (Reliability)	Quando un cliente effettua un ordine, si aspetta che venga elaborato correttamente .	Porterà a transazioni distribuite, Saga (Lez. 12), Outbox pattern, idempotenza.

Il collegamento alla lezione successiva

Con il problema chiaro e i requisiti definiti, il passo successivo del ciclo è **LEARN**: si studierà **l'approccio architetturale più ovvio e immediato — l'architettura monolitica** — che è appunto la Lezione 2.